SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: **Database Management Systems**

COURSE CODE: **CSA0504**

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Pseudocode Writing Task (Algorithm Design) (Medium)

Assessment Tool Weightage: 35%

QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.	BL3 – Apply	**5**
2	Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.	BL4 – Analyze	**5**
3	Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.	BL3 – Apply	**5**
4	Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.	BL4 – Analyze	**5**

5	Design a pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.	BL3 – Apply	**5**
6	Write pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.	BL4 – Analyze	**5**
7	Write a pseudocode algorithm for the ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) recovery technique with Analysis, Redo, and Undo phases.	BL3 – Apply	**5**
8	Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.	BL3 – Apply	**5**
9	Write a pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.	BL3 – Apply	**5**
10	Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.	BL4 – Analyze	**5**

Code of Conduct:

I B Aishwarya/192421432 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: B Aishwarya

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

1)Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.

ANS

ALGORITHM Query_Processing

INPUT: SQL Query

OUTPUT: Query Result

Step 1: Receive SQL Query

READ SQL_Query

Step 2: Lexical Analysis

Divide SQL_Query into tokens

Identify keywords, table names, column names,

operators, and values

Step 3: Syntax Analysis (Parsing)

Check whether SQL syntax is correct

IF syntax is incorrect THEN

DISPLAY "Syntax Error"

STOP

END IF

Step 4: Create Parse Tree

Convert the SQL query into a Parse Tree

Step 5: Semantic Analysis

Check whether tables and columns exist

Check data types and constraints

IF semantic error THEN

DISPLAY "Semantic Error"

STOP

END IF

Step 6: Convert to Relational Algebra

Convert the Parse Tree into a relational algebra expression

Step 7: Query Optimization

Apply optimization techniques:

- Push selections early
- Perform projections early
- Choose efficient join order
- Select suitable indexes

Generate an optimized query plan

Step 8: Query Execution

Send the optimized query plan to the execution engine

Access required tables and indexes

Perform selections, projections, joins, sorting, etc.

Step 9: Generate Result

Collect the output tuples

Format the result according to the SQL query

Step 10: Display Result

DISPLAY Query Result

END ALGORITHM

2) Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.

ANS

```
ALGORITHM Cost_Based_Join_Optimization
INPUT:
    Relations R1, R2, ..., Rn
    Join conditions
    Number of pages in each relation
    Number of tuples in each relation
OUTPUT:
    Best join order with minimum I/O cost

Step 1: Get all possible join orderings
    Generate different join orders for the relations

Step 2: Set minimum cost
    MIN_COST ← Infinity
    BEST_ORDER ← NULL

Step 3: For each join order
    FOR each ORDER in Join_Orders DO

        COST ← 0
        RESULT_SIZE ← Size of first relation

        FOR each JOIN in ORDER DO

            Estimate the size of the intermediate result
```

```
        Calculate I/O cost of the join

        COST ← COST + Join_IO_Cost

        RESULT_SIZE ← Estimated intermediate result size

      END FOR

      Step 4: Compare costs
      IF COST < MIN_COST THEN
        MIN_COST ← COST
        BEST_ORDER ← ORDER
      END IF

    END FOR

Step 5: Return the best plan
    DISPLAY BEST_ORDER
    DISPLAY MIN_COST

END ALGORITHM
```

3) Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.

ANS

```
ALGORITHM Nested_Loop_Join(R, S)
INPUT:
   Two relations R and S

OUTPUT:
   Joined relation

FOR each tuple r in R DO
   FOR each tuple s in S DO
      IF r.join_attribute = s.join_attribute THEN
         ADD (r, s) to Result
      END IF
   END FOR
END FOR

RETURN Result

END ALGORITHM
```

4) Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.

ANS

```
ALGORITHM Detect_Deadlock(WFG)

INPUT:
   Wait-For Graph containing transactions as nodes
   and waiting relationships as edges

OUTPUT:
   Deadlock detected or no deadlock

Step 1: Create the Wait-For Graph
   Create one node for each transaction

Step 2: Add waiting edges
   FOR each waiting transaction Ti DO
      IF Ti is waiting for Tj THEN
         Add edge Ti → Tj
      END IF
   END FOR

Step 3: Check for a cycle
   FOR each transaction T DO
      Mark T as NOT_VISITED
   END FOR

   FOR each transaction T DO
      IF T is NOT_VISITED THEN
```

```
        IF DFS(T) finds a cycle THEN
            DISPLAY "Deadlock Detected"
            RETURN
        END IF
    END IF
END FOR

Step 4:
    DISPLAY "No Deadlock"

END ALGORITHM
```

5) Design a pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.

ANS

```
ALGORITHM Basic_2PL(Transaction T)

BEGIN TRANSACTION

Phase 1: Growing Phase

    FOR each data item X required by T DO

        IF T needs to READ X THEN
            REQUEST Shared_Lock(X)
        ELSE IF T needs to WRITE X THEN
            REQUEST Exclusive_Lock(X)
        END IF

        IF Lock(X) is available THEN
```

GRANT Lock to T

ELSE

WAIT until Lock(X) becomes available

END IF

END FOR

Phase 2: Shrinking Phase

When T completes all lock requests:

FOR each locked data item X DO

RELEASE Lock(X)

END FOR

COMMIT TRANSACTION

END ALGORITHM

6) Write pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.

ANS

ALGORITHM Thomas_Write_Rule(T, X, Operation)

INPUT:

Transaction T

Data item X

Operation = READ or WRITE

Each transaction T has:

TS(T) = Timestamp of T

Each data item X has:

R_TS(X) = Largest timestamp of a transaction that read X

W_TS(X) = Largest timestamp of a transaction that wrote X

```
IF Operation = READ(X) THEN

    IF TS(T) < W_TS(X) THEN
        ABORT T
        DISPLAY "Read rejected"
    ELSE
        PERFORM READ(X)
        R_TS(X) = MAX(R_TS(X), TS(T))
        DISPLAY "Read successful"
    END IF


ELSE IF Operation = WRITE(X) THEN

    IF TS(T) < R_TS(X) THEN
        ABORT T
        DISPLAY "Write rejected"

    ELSE IF TS(T) < W_TS(X) THEN
        IGNORE WRITE(X)
        DISPLAY "Obsolete write ignored using Thomas' Write Rule"

    ELSE
```

```
        PERFORM WRITE(X)
        W_TS(X) = TS(T)
        DISPLAY "Write successful"
    END IF

END IF

END ALGORITHM
```

7) Write a pseudocode algorithm for the ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) recovery technique with Analysis, Redo, and Undo phases.

ANS

```
ALGORITHM ARIES_Recovery(Log)
INPUT:
    Log records stored on disk
OUTPUT:
    Database recovered to a consistent state
PHASE 1: ANALYSIS

    Read the log from the last checkpoint

    Create Transaction Table
    Create Dirty Page Table

    FOR each log record L DO

        IF L belongs to a transaction T THEN
            Add T to Transaction Table if not present
            Update T's status
        END IF
```

```
        IF L represents a modified page P THEN
            Add P to Dirty Page Table if not present
        END IF

    END FOR

    Identify:
        - Winner transactions = committed transactions
        - Loser transactions = uncommitted transactions
PHASE 2: REDO

    Find the smallest Recovery LSN from the
    Dirty Page Table

    FOR each log record L from Recovery LSN onward DO

        IF L represents a database update THEN

            IF the update is not already present in the page THEN
                REDO the update
            END IF

        END IF

    END FOR
PHASE 3: UNDO

    FOR each loser transaction T DO
```

```
        Start from the last log record of T

        WHILE T has undoable log records DO

            Read the log record L

            UNDO the operation recorded in L

            Write a compensation log record (CLR)

            Move to the previous log record of T

        END WHILE

        Write an END record for T

    END FOR


    Write all recovery information to disk

    DISPLAY "Database Recovery Completed"

END ALGORITHM
```

8) Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.

ANS

ALGORITHM Shadow_Paging_Recovery

INPUT:

Database pages

Transaction operations

OUTPUT:

Recovered database

Step 1: Start Transaction

Create Shadow Page Table

Current Page Table ← Copy of Shadow Page Table

Step 2: Perform Updates

FOR each update to page P DO

Create a new copy of page P

Modify the new page

Update Current Page Table

to point to the new page

END FOR

Step 3: Commit

IF transaction completes successfully THEN

Write Current Page Table to disk

Make Current Page Table
the new Shadow Page Table

DISPLAY "Transaction Committed"

ELSE

GOTO ROLLBACK
END IF

Step 4: Failure / Rollback

ROLLBACK:

Discard Current Page Table

Restore Shadow Page Table
as the Current Page Table

Discard all newly modified pages

DISPLAY "Transaction Rolled Back"

Step 5: End

Return recovered database

END ALGORITHM

9) Write a pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.

ANS

ALGORITHM Immediate_Update_Recovery(Log)

INPUT:

Log containing transaction operations

OUTPUT:

Recovered and consistent database

PHASE 1: ANALYSIS

Read the log from the beginning

Committed ← empty set

Active ← empty set

FOR each log record L DO

IF L is START(T) THEN

Add T to Active

```
        ELSE IF L is COMMIT(T) THEN
            Add T to Committed
            Remove T from Active

        END IF

    END FOR


PHASE 2: REDO

    FOR each log record L in forward order DO

        IF L is an UPDATE record THEN
            Perform the new value of the update
        END IF

    END FOR


PHASE 3: UNDO

    FOR each transaction T in Active DO

        Scan T's log records in reverse order

        FOR each UPDATE record of T DO
            Restore the old value
```

END FOR

Write ABORT(T) to the log

END FOR

Write all recovered pages to disk

DISPLAY "Recovery Completed"

END ALGORITHM

10) Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.

ANS

ALGORITHM Database_Connectivity_Manager

INPUT:

SQL Query

Query Parameters

OUTPUT:

Query Result

Step 1: Initialize Connection Pool

Create a pool of database connections

Set maximum number of connections

WHILE application is running DO

Step 2: Get Connection

TRY

Get an available connection from the pool

IF connection is not available THEN

WAIT until a connection becomes available

END IF

Step 3: Start Transaction

BEGIN TRANSACTION

Step 4: Execute Query

Prepare SQL Query

Set Query Parameters

Execute SQL Query

Store Query Result

Step 5: Commit

COMMIT TRANSACTION

Return Result

EXCEPTION

Step 6: Rollback

ROLLBACK TRANSACTION

DISPLAY "Error occurred. Transaction rolled back."

FINALLY

Step 7: Return Connection

Return connection to the connection pool

END TRY

END WHILE

END ALGORITHM